GUÍA DE LABORATORIO

DevSecOps: Pipeline CI/CD con GitHub Actions

UNIDAD	SESION	TIPO	HERRAMIENTAS
3	9	LABORATORIO	GitHub Actions · Semgrep · pip-audit

CUGDL — Universidad de Guadalajara
Arquitectura de Sistemas de Seguridad
Licenciatura en Ciberseguridad

Autor: Carlos Pulido Rosas
Abril 2026

CONTEXTO	PASOS DEL LAB	ENTREGABLE
Hacer fork del repositorio del docente, que ya contiene una aplicacion Flask con vulnerabilidades intencionales.	1. Fork del repositorio base 2. Explorar el codigo vulnerable 3. Crear el pipeline de seguridad	Capturas de pantalla de cada LAB + URL del repositorio forkeado + reporte de vulnerabilidades + reflexión en un solo PDF.

OBJETIVOS DEL LABORATORIO

1. Hacer fork de un repositorio existente y entender la estructura de una aplicacion web con vulnerabilidades intencionales.
2. Construir un pipeline CI/CD de seguridad desde cero en GitHub Actions con SAST (Semgrep) y analisis de dependencias (pip-audit).
3. Verificar que el pipeline detecta automaticamente vulnerabilidades reales — SQL injection, command injection, credenciales hardcodeadas y CVEs en dependencias.
4. Interpretar los resultados del pipeline y documentar cada vulnerabilidad detectada con su Rule ID, severidad, riesgo y solucion.
5. (Reto) Intentar introducir una vulnerabilidad que evada la deteccion del pipeline y luego mejorar las reglas para detectarla.

LABORATORIO

⚠

Antes de empezar

- Necesitas una cuenta gratuita en GitHub — crear en https://github.com/signup si no tienes.
- Este laboratorio se realiza completamente en el navegador — no requiere instalar nada en tu computadora.
- El repositorio base del docente esta disponible en el repositorio del curso en GitHub.
- GitHub Actions tiene 2,000 minutos gratuitos al mes — suficiente para todo el semestre.

📖

CONTEXTO

El repositorio base ya contiene una aplicacion Flask con vulnerabilidades intencionales que representan errores comunes en aplicaciones de produccion real. Tu trabajo es configurar el pipeline de seguridad que las detecte automaticamente — exactamente como lo hace un equipo DevSecOps en la industria.

Cuando termines tendras un repositorio GitHub propio con un pipeline de seguridad funcional que puedes mostrar como evidencia de experiencia practica en DevSecOps.

LAB 1

Fork del repositorio base

El fork crea una copia del repositorio del docente en tu cuenta de GitHub. A partir de ese momento trabajas en tu propia copia — tus cambios no afectan el repositorio original.

```
# Paso 1 — Ir al repositorio del curso en GitHub:
# https://github.com/Carpuro/lc_arquitectura_de_sistemas_de_seguridad

# Paso 2 — Navegar a la carpeta del laboratorio:
# Unidad_3/devsecops-lab/

# Paso 3 — Hacer fork del repositorio COMPLETO:
# Clic en el boton 'Fork' (esquina superior derecha)
# Owner: tu usuario de GitHub
# Repository name: devsecops-lab-[tu-nombre]
# Desmarcar: 'Copy the main branch only'
# Clic en 'Create fork'

# Paso 4 — Verificar que el fork fue exitoso:
# Debes ver: tu-usuario/devsecops-lab-[tu-nombre]
# Con el mensaje: 'forked from Carpuro/lc_arquitectura...'

# Paso 5 — Habilitar GitHub Actions en tu fork (OBLIGATORIO):
# GitHub deshabilita los workflows automáticamente en todos los forks.
# Sin este paso el pipeline NUNCA se ejecutará.
#
#   1. Ve a la pestaña Actions de TU fork (no el del docente)
#   2. Haz clic en el botón verde:
#      "I understand my workflows, go ahead and enable them"
#   3. Listo: cada commit dispara el pipeline automáticamente
#
# Paso 6 — Navegar dentro del fork a la carpeta del lab:
# Unidad_3/devsecops-lab/
# Aqui esta el codigo que vas a analizar y el pipeline que vas a construir
```

Diferencia entre fork y clone

- Fork: crea una copia del repositorio en tu cuenta de GitHub — todo ocurre en la nube, sin descargar nada.
- Clone: descarga el repositorio a tu computadora para trabajar localmente.
- En este laboratorio usamos fork porque todo el trabajo ocurre en la interfaz web de GitHub y en los servidores de GitHub Actions — no necesitas tener Git instalado.

LAB 2

Explorar el codigo vulnerable y entender que tiene

Antes de configurar el pipeline, debes entender que vulnerabilidades tiene el codigo. Leer el codigo con perspectiva de seguridad es la habilidad mas importante de esta sesion.

Paso 2.1 — Abrir app/app.py en el navegador y analizar el codigo:

```
# El archivo app/app.py contiene estas vulnerabilidades intencionales.
# Para cada una, antes de ver los resultados del pipeline, responde:

# - Por que es peligrosa?
# - Como la explotaria un atacante?
# - Como se corrige?

# VULNERABILIDAD 1 — Linea con DB_PASSWORD y SECRET_KEY
# Tipo: Hardcoded Secret
# Que hace: expone credenciales directamente en el codigo fuente
# Riesgo: cualquier persona con acceso al repositorio obtiene la credencial

# VULNERABILIDAD 2 — Funcion get_user(), linea con f-string en query SQL
# Tipo: SQL Injection
# Que hace: concatena el input del usuario directamente en la query SQL
# Payload de ataque: ?username=' OR '1'='1
# Efecto: devuelve todos los usuarios de la base de datos

# VULNERABILIDAD 3 — Funcion ping(), linea con subprocess.run()
# Tipo: Command Injection
# Que hace: pasa el input del usuario directamente al shell del sistema
# Payload de ataque: ?host=8.8.8.8; cat /etc/passwd
# Efecto: ejecuta comandos arbitrarios en el servidor

# VULNERABILIDAD 4 — app.run(debug=True)
# Tipo: Debug mode en produccion
# Que hace: expone el debugger interactivo y stack traces completos
# Riesgo: el atacante puede ejecutar codigo Python arbitrario via debugger

# VULNERABILIDAD 5 — Función read_logs(), endpoint /logs
# Tipo: Path Traversal
# Qué hace: recibe el nombre de un archivo y lo lee del sistema sin validar ruta
# Payload: ?file=../../etc/passwd
# Efecto: permite leer archivos arbitrarios del servidor
```

Paso 2.2 — Revisar app/requirements.txt:

```
# El archivo requirements.txt tiene versiones antiguas con CVEs conocidos.
# Anotar las versiones antes de ver lo que detecta Dependabot:

Flask==2.0.1
Werkzeug==2.0.1
Jinja2==3.0.1
cryptography==38.0.0    # CVE-2023-0286 CVSS 7.4
requests==2.25.0        # tiene actualizaciones de seguridad disponibles
```

LAB 3

Construir el pipeline de seguridad en GitHub Actions

El archivo .github/workflows/security.yml ya existe en tu fork pero esta vacio — solo tiene comentarios con instrucciones. Tu trabajo es completarlo con el pipeline de seguridad.

Paso 3.1 — Abrir el archivo en la interfaz web de GitHub:

```
# En tu fork, navegar a:
# .github/workflows/security.yml

# Clic en el icono de lapiz (editar archivo)
# Verás el archivo vacio con comentarios de instrucciones
# El archivo ya contiene un esqueleto con marcadores TODO. Completa únicamente los
# bloques TODO siguiendo los comentarios. No modif iques el resto.
```

Paso 3.2 — Ejemplos para cada Etapa

```
    # == ETAPA 1: SAST con Semgrep ==
  sast:
    name: 'SAST - Semgrep'
    runs-on: ubuntu-latest
    steps:
      - name: Checkout codigo
        uses: actions/checkout@v4

      - name: Instalar Semgrep
        run: pip install semgrep

      - name: Ejecutar analisis SAST
        run: |
          semgrep --config p/python \
                  --config p/owasp-top-ten \
                  --config p/secrets \
                  --config p/sql-injection \
                  --error \
                  --json \
                  --output=semgrep-results.json \
                  Unidad_3_Seguridad_en_Aplicaciones_y_Datos/devsecops-lab/app/ ||
true
          semgrep --config p/python \
                  --config p/owasp-top-ten \
                  --config p/secrets \
                  --config p/sql-injection \
                  Unidad_3_Seguridad_en_Aplicaciones_y_Datos/devsecops-lab/app/

      - name: Guardar resultados SAST
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: semgrep-results
          path: semgrep-results.json

  # == ETAPA 2: Analisis de dependencias ==
  sca:
    name: 'SCA - pip-audit'
    runs-on: ubuntu-latest
    steps:
      - name: Checkout codigo
        uses: actions/checkout@v4
```

```
      - name: Configurar Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Instalar y ejecutar pip-audit
        run: |
          pip install pip-audit
          pip-audit -r Unidad_3_Seguridad_en_Aplicaciones_y_Datos/devsecops-
lab/app/requirements.txt \
                    --format=json \
                    --output=audit-report.json || true
          echo '--- Reporte de CVEs ---'
          pip-audit -r Unidad_3_Seguridad_en_Aplicaciones_y_Datos/devsecops-
lab/app/requirements.txt

      - name: Guardar reporte de auditoria
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: pip-audit-report
          path: audit-report.json

  # == ETAPA 3: Tests ==
  tests:
    name: 'Tests - pytest'
    runs-on: ubuntu-latest
    needs: [sast, sca]
    steps:
      - name: Checkout codigo
        uses: actions/checkout@v4

      - name: Configurar Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Instalar dependencias
        run: |
          pip install -r Unidad_3_Seguridad_en_Aplicaciones_y_Datos/devsecops-
lab/app/requirements.txt
          pip install pytest

      - name: Ejecutar tests
        run: pytest Unidad_3_Seguridad_en_Aplicaciones_y_Datos/devsecops-
lab/tests/ -v

  # == ESTADO FINAL DEL PIPELINE ==
  pipeline-status:
    name: 'Estado final del pipeline'
    runs-on: ubuntu-latest
    needs: [sast, sca, tests]
    if: always()
    steps:
      - name: Resumen
        run: |
          echo 'SAST:         ${{ needs.sast.result }}'
          echo 'Dependencias: ${{ needs.sca.result }}'
          echo 'Tests:        ${{ needs.tests.result }}'
```

Paso 3.3 — Guardar el archivo y ver el pipeline ejecutarse:

```
# En GitHub, despues de pegar el contenido:
# 1. Scroll hacia abajo hasta 'Commit changes'
# 2. Mensaje de commit: 'feat: agregar pipeline de seguridad CI/CD'
# 3. Clic en 'Commit changes'

# ANTES de hacer commit: asegúrate de haber habilitado
# Actions en tu fork (Paso 5). Sin eso el pipeline no corre.
# También puedes disparar el pipeline manualmente con
#   "Run workflow" en la pestaña Actions (sin hacer commit).
#
# El pipeline se dispara automaticamente con el commit.
# Para verlo:
# Ir a la pestana 'Actions' de tu repositorio
# Clic en 'Pipeline de Seguridad DevSecOps'
# Clic en el workflow mas reciente

# Resultado esperado del pipeline:
#   SAST - Semgrep  → ROJO   (detectó vulnerabilidades = correcto)
#   SCA - pip-audit → VERDE  (muestra CVEs, no bloquea el pipeline)
#   Tests - pytest  → VERDE  (corren aunque SAST falle)
#
# Si SAST aparece rojo: el pipeline funciona correctamente.
# Los tests siempre corren gracias a continue-on-error en el job sast.
#
# Etapas que verás ejecutarse:
# SAST - Semgrep     → SAST - Semgrep
# SCA - Dependencias y CVEs    → SCA pip-audit
# Tests de la aplicacion       → Test - pytest

# El pipeline tarda aproximadamente 2-3 minutos en completarse
```

LAB 4

Activar Dependabot y Secret Scanning

Dependabot monitorea las dependencias y abre Pull Requests automaticos cuando detecta CVEs. Secret Scanning detecta credenciales en el codigo fuente.

```
# Activar desde la interfaz de GitHub:

# 1. Ir a tu repositorio → Settings
# 2. En el menu lateral izquierdo Security and quality → Advanced Security
# 3. Activar los siguientes controles:

#    Dependabot alerts:           ON
#    Dependabot security updates: ON
#    Secret scanning:             ON
#    Push protection:             ON

# 4. Ir a Security → Dependabot alerts
#    Verás las alertas de CVE para las dependencias de requirements.txt
#    Cada alerta muestra: CVE ID, severidad CVSS, descripcion y fix sugerido

# 5. Ir a Security → Secret scanning alerts
#    Si detecto DB_PASSWORD o SECRET_KEY en app.py, aparecerán aquí
#    GitHub bloquea futuros pushes que contengan el mismo patron de secreto

# Tomar screenshot de cada seccion — son parte del entregable
```

LAB 5

Analizar las detecciones y documentar las vulnerabilidades

Revisamos los resultados del pipeline etapa por etapa. Para cada vulnerabilidad detectada documentamos: que es, por que es peligrosa y como se corrige.

Paso 5.1 — Analizar resultados de Semgrep:

```
# En Actions → workflow mas reciente → clic en 'SAST - Semgrep'
# Verás la salida de Semgrep con las vulnerabilidades detectadas

# Para cada vulnerabilidad que aparezca, completar esta tabla:

Vulnerabilidad 1:
  Rule ID:       ______________________
  Severidad:     ______________________
  Archivo:       ______________________
  Linea:         ______________________
  Descripcion:   ______________________
  Riesgo real:   ______________________
  Correccion:    ______________________

Vulnerabilidad 2:
  Rule ID:       ______________________
  Severidad:     ______________________
  Archivo:       ______________________
  Linea:         ______________________
  Descripcion:   ______________________
  Riesgo real:   ______________________
  Correccion:    ______________________

# Continuar para todas las vulnerabilidades detectadas
# Semgrep deberia detectar al menos 3 problemas distintos
```

Paso 5.2 — Analizar resultados de pip-audit:

```
# En Actions → workflow → 'SCA - Dependencias y CVEs'
# También descargar el artifact 'pip-audit-report' para el reporte JSON

# Para cada CVE detectado, completar:

CVE 1:
  Paquete:          ______________________
  Version actual:   ______________________
  CVE ID:           ______________________
  Severidad CVSS:   ______________________
  Descripcion:      ______________________
  Version segura:   ______________________

# En Dependabot alerts (Security tab) verás las mismas alertas
# con mas detalle incluyendo el Pull Request sugerido para actualizar
```

Paso 5.3 — Descargar y revisar los artifacts:

```
# Los artifacts son los reportes generados por el pipeline:

# En Actions → workflow → scroll hasta 'Artifacts'
# Descargar:
#   semgrep-results    → reporte JSON de Semgrep
#   pip-audit-report   → reporte JSON de pip-audit

# Estos archivos JSON contienen el reporte completo en formato
# que se puede integrar con herramientas de gestion de vulnerabilidades
# como Jira, GitHub Issues o dashboards de seguridad

# Incluir los archivos JSON descargados en el entregable final
```

ENTREGABLE

- URL del repositorio: enlace a tu fork en GitHub con el pipeline configurado y funcionando.
- screenshot_pipeline_acciones — pestana Actions mostrando el workflow con sus etapas.
- screenshot_sast_resultados — salida de Semgrep con las vulnerabilidades detectadas y sus Rule IDs.
- screenshot_sca_resultados — salida de pip-audit con los CVEs detectados en las dependencias.
- screenshot_dependabot — pestana Security mostrando las alertas de Dependabot.
- screenshot_pipeline_fallido — el pipeline en estado de fallo por las vulnerabilidades.
- semgrep-results.json y audit-report.json — artifacts descargados del pipeline.
- reporte_vulnerabilidades— tabla con todas las vulnerabilidades detectadas completada en el Paso 5.
- reflexion — respuestas a las 3 preguntas de reflexion
- **Nombre del archivo:** `Apellidos_Nombre(s)_Lab4.pdf`

Plazo: Fecha en Classroom. Tardio: maximo 85/100.

PREGUNTAS DE REFLEXION

1. En el Lab 5 Semgrep detecto las vulnerabilidades en el codigo. Sin embargo, SAST no puede detectar vulnerabilidades que solo aparecen en tiempo de ejecución, por ejemplo, una falla en la logica de autorizacion que permite a un usuario ver los datos de otro usuario del mismo nivel. Da un ejemplo concreto de una vulnerabilidad en app.py que Semgrep no podria detectar y que requeriria DAST o una revision manual para encontrarla.
2. El pipeline esta configurado para mostrar las vulnerabilidades pero sin bloquear el merge (--error esta comentado). En un equipo de desarrollo real, que criterios

usarias para decidir que nivel de severidad de Semgrep bloquea el merge automaticamente vs. solo genera una advertencia? Considera el impacto en la velocidad del equipo y el riesgo de security fatigue.

3. pip-audit detecto CVEs en las dependencias del proyecto. En la practica, actualizar una dependencia puede romper la aplicacion si hay cambios incompatibles entre versiones (breaking changes). Describe el proceso que seguirias para actualizar cryptography de 38.0.0 a la version segura en un proyecto de produccion sin interrumpir el servicio ni introducir regresiones.

RECURSOS

- [Docs] GitHub Actions — https://docs.github.com/en/actions — documentacion oficial completa
- [Docs] Semgrep Rules — https://semgrep.dev/r — explorador de reglas con descripcion y ejemplos
- [Docs] pip-audit — https://pypi.org/project/pip-audit/ — herramienta de auditoria de dependencias Python
- [Docs] Dependabot — https://docs.github.com/en/code-security/dependabot — configuracion y uso
- [Docs] OWASP Top 10 — https://owasp.org/www-project-top-ten/ — referencia de vulnerabilidades
- [TryHackMe] OWASP Top 10 — https://tryhackme.com/room/owasptop10 — practica de las vulnerabilidades detectadas en este lab